

# AI ASSISTED CODING

SUMANTH POLAM

2303A51121

BATCH – 03

07 – 04 – 2026

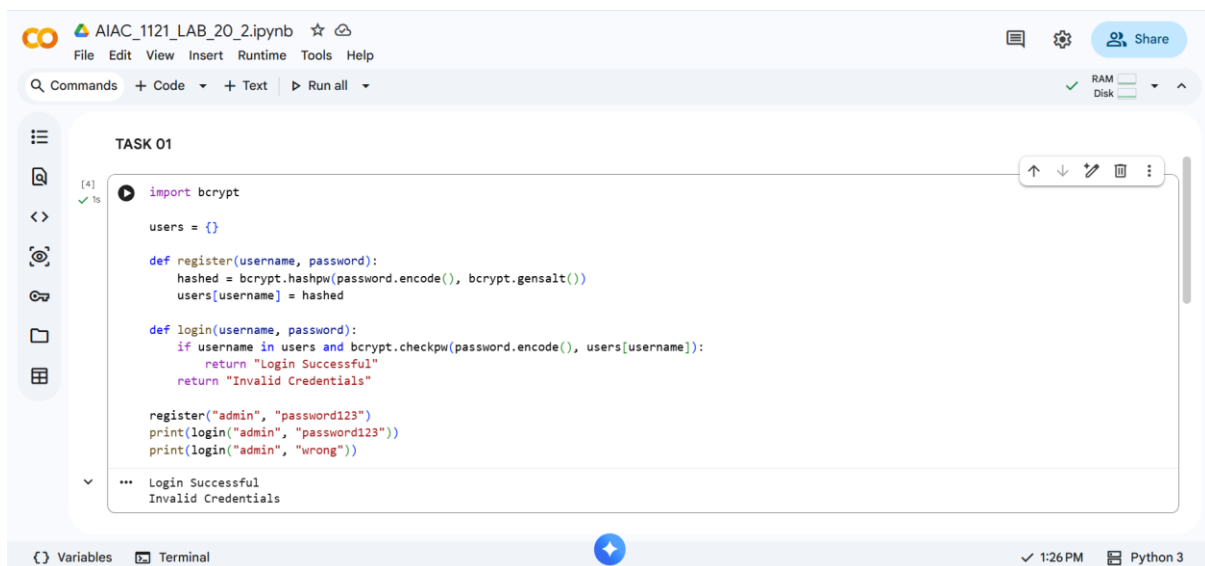
## ASSIGNMENT – 20.2

**Lab 20 : Security Testing : Identifying Vulnerabilities in AI – Generated Code.**

**Task – 01 : Password Security Enhancement.**

**Prompt :** Generate a Python user registration and login script that stores usernames and passwords.

**Code & Output :**



The screenshot shows a Jupyter Notebook titled "AIAC\_1121\_LAB\_20\_2.ipynb". The code is written in a cell labeled "TASK 01". The code imports the bcrypt module and creates a dictionary to store users. It defines two functions: register and login. The register function takes a username and password, hashes the password using bcrypt, and stores the username and hashed password in the dictionary. The login function takes a username and password, checks if the username is in the dictionary and if the password matches the hashed password, and returns "Login Successful" or "Invalid Credentials". The code also includes test calls for both functions.

```
import bcrypt

users = {}

def register(username, password):
    hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
    users[username] = hashed

def login(username, password):
    if username in users and bcrypt.checkpw(password.encode(), users[username]):
        return "Login Successful"
    return "Invalid Credentials"

register("admin", "password123")
print(login("admin", "password123"))
print(login("admin", "wrong"))
```

The output of the code is shown below the code cell:

```
*** Login Successful
Invalid Credentials
```

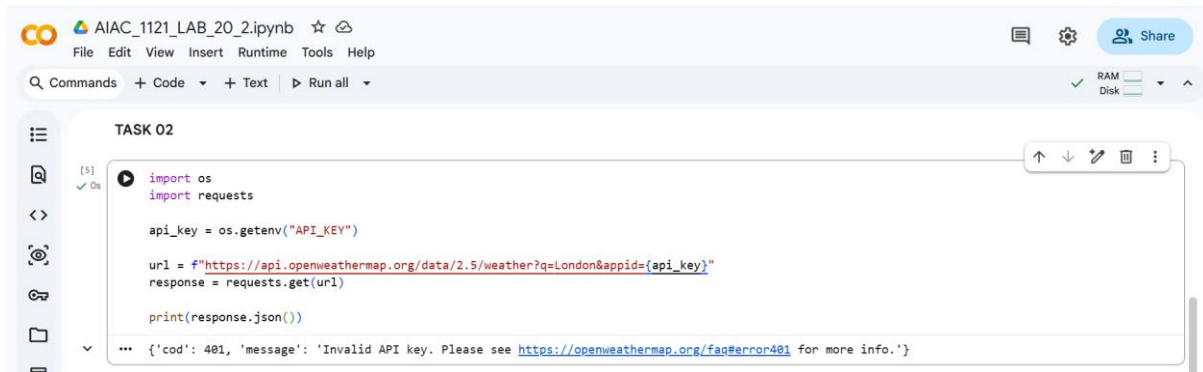
**Explanation :**

Plain text passwords are unsafe because they can be easily exposed in data breaches. Hashing converts passwords into irreversible values, improving security. bcrypt adds salting, preventing attacks like rainbow tables. The updated script securely verifies login without storing real passwords.

## Task – 02 : Secure API Key Management.

**Prompt :** Generate a Python script that fetches data from an API using an API key.

**Code :**



```
AIAC_1121_LAB_20_2.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
TASK 02
import os
import requests

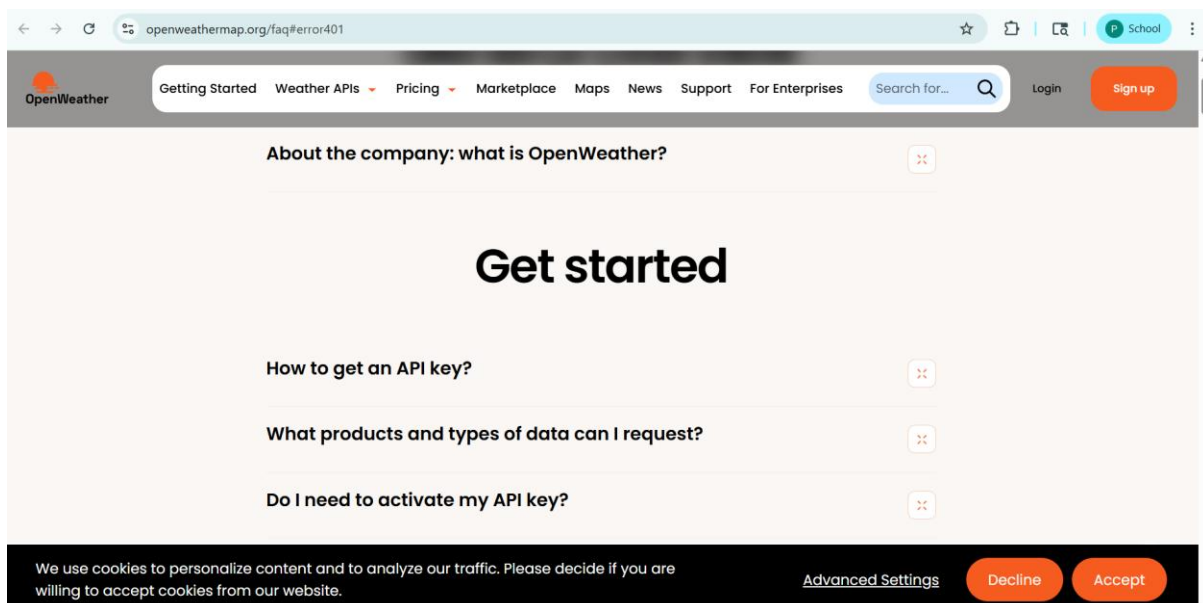
api_key = os.getenv("API_KEY")

url = f"https://api.openweathermap.org/data/2.5/weather?q=London&appid={api_key}"
response = requests.get(url)

print(response.json())

... {'cod': 401, 'message': 'Invalid API key. Please see https://openweathermap.org/faq#error401 for more info.'}
```

**Output :**



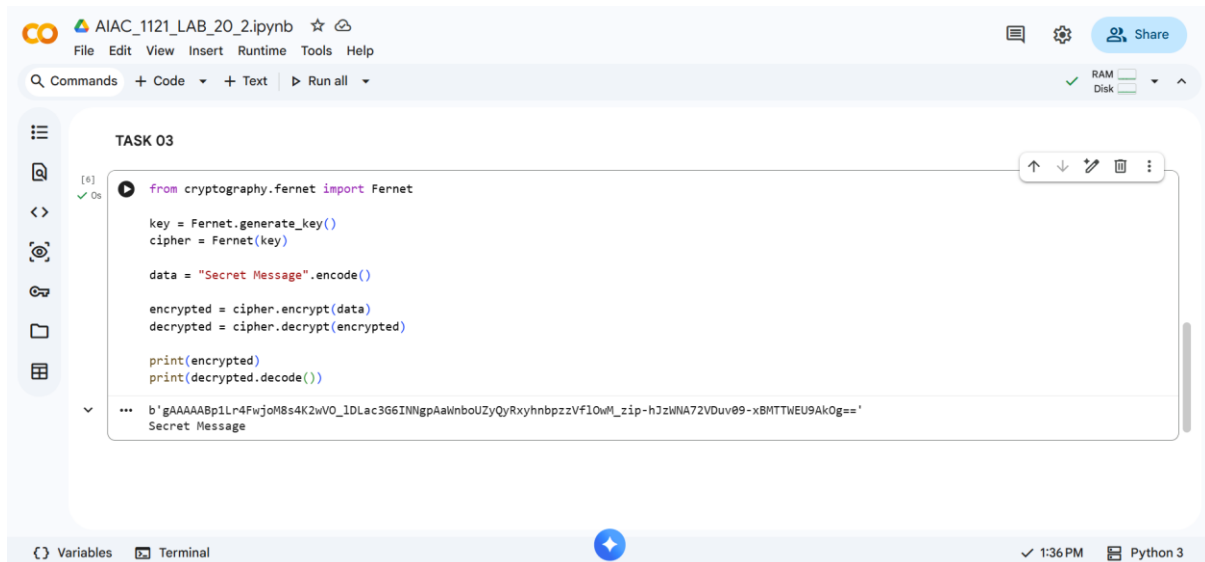
**Explanation :**

Hardcoding API keys exposes them in source code and version control. Environment variables securely store sensitive data outside the code. This prevents unauthorized access and improves deployment security. The script still works normally while keeping credentials hidden.

### Task – 03 : Weak Encryption Detection.

**Prompt :** Generate a Python encryption-decryption program using a simple method.

#### Code & Output :



The screenshot shows a Jupyter Notebook interface with a file named 'AIAC\_1121\_LAB\_20\_2.ipynb'. The notebook contains a single cell titled 'TASK 03' with the following Python code:

```
[6]: from cryptography.fernet import Fernet

key = Fernet.generate_key()
cipher = Fernet(key)

data = "Secret Message".encode()

encrypted = cipher.encrypt(data)
decrypted = cipher.decrypt(encrypted)

print(encrypted)
print(decrypted.decode())
```

The output of the code is displayed below the cell:

```
... b'gAAAAABp1Lr4Fwj0M8s4K2wVO_IDLac3G6INNgpAaWnboUZyQyRxyhnbpzVf10wM_zip-hJzWNA72VDuv09-xBMTTWEU9AkOg=='
Secret Message
```

The interface also shows a 'Variables' tab at the bottom left and a 'Terminal' tab at the bottom right. The status bar at the bottom indicates '1:36 PM' and 'Python 3'.

#### Explanation :

Basic encoding like base64 is reversible and not secure encryption. Fernet provides strong symmetric encryption using secure algorithms. Data cannot be decrypted without the correct key. This ensures confidentiality and protects sensitive information.

### Task – 04 : Real Time Application : Security Review of AI – Generated Web Service.

**Prompt :** Generate a simple Flask API for user login.

## Code & Output :



A screenshot of a Jupyter Notebook interface. The top bar shows the file name 'AIAC\_1121\_LAB\_20\_2.ipynb' and a 'Share' button. The menu bar includes 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a search bar and a 'Run all' button. The left sidebar contains icons for file explorer, search, and other notebook functions. The main area displays Python code for a Flask login application. The code imports Flask, request, jsonify, and bcrypt. It defines a Flask app, a dictionary of users with hashed passwords, and a login endpoint that checks the provided username and password. The terminal output at the bottom shows the app running on http://127.0.0.1:5000 with a warning to use a production server.

```
[9] from flask import Flask, request, jsonify
import bcrypt

app = Flask(__name__)
users = {
    "admin": bcrypt.hashpw("password123".encode(), bcrypt.gensalt())
}
@app.route('/login', methods=['POST'])
def login():
    data = request.json
    username = data.get("username")
    password = data.get("password")
    if username in users and bcrypt.checkpw(password.encode(), users[username]):
        return jsonify({"message": "Success"})
    return jsonify({"message": "Unauthorized"}), 401

if __name__ == '__main__':
    app.run()

... * Serving Flask app '__main__'
* Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
```

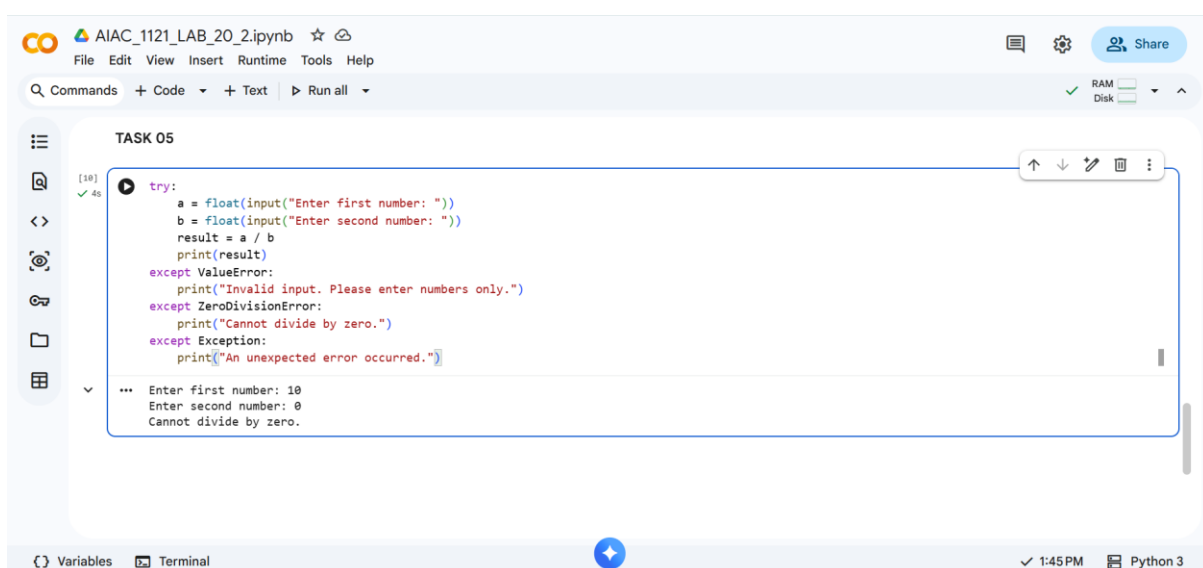
## Explanation :

Common vulnerabilities include plain-text passwords, no validation, and insecure responses. bcrypt ensures passwords are securely stored and verified. Input is handled safely using JSON parsing. The API now reduces authentication risks and improves overall security.

## Task – 05 : Exception Handling Improvement.

**Prompt :** Generate a Python program that takes two numbers and performs division.

## Code & Output :



A screenshot of a Jupyter Notebook interface. The top bar shows the file name 'AIAC\_1121\_LAB\_20\_2.ipynb' and a 'Share' button. The menu bar includes 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a search bar and a 'Run all' button. The left sidebar contains icons for file explorer, search, and other notebook functions. The main area displays Python code for a division program with exception handling. The code prompts the user for two numbers and attempts to divide them. It catches ValueError for non-numeric input and ZeroDivisionError for division by zero. The terminal output shows the program running and handling these exceptions correctly.

```
TASK 05

[10] try:
[45]     a = float(input("Enter first number: "))
        b = float(input("Enter second number: "))
        result = a / b
        print(result)
    except ValueError:
        print("Invalid input. Please enter numbers only.")
    except ZeroDivisionError:
        print("Cannot divide by zero.")
    except Exception:
        print("An unexpected error occurred.")

... Enter first number: 10
Enter second number: 0
Cannot divide by zero.
```

**Explanation :**

Programs without exception handling may crash during runtime errors. Invalid inputs and division by zero are common issues. try-except blocks ensure smooth execution and prevent crashes. Users receive clear error messages instead of program failure.

**THANK YOU!!**